

# Click-to-Source 3D

## Stage 2 Architectural Validation Report Metadata Convention & Future Instrumentation Design

### 1. Executive Summary

Stage 1 answered a technical feasibility question: can a runtime Three.js / React Three Fiber object reliably carry provenance information through the R3F lifecycle. Stage 2 answers a different kind of question: given that it can, what is the one correct way to do it, and where does that decision stop applying.

Stage 2 converted Stage 1's experimental findings into a permanent architectural contract. No runtime functionality was added and no production code was written during this stage. The objective was narrower and more disciplined than implementation: eliminate architectural ambiguity before Stage 3 begins building the core engine and overlay on top of it.

This report documents how that contract was reached, what evidence supports each decision, and — equally important — which decisions were deliberately left open rather than guessed at.

### 2. Background

Stage 1 (see the Stage 1 Experimental Report) ran seven experiments and established, with raw runtime logs rather than assumption:

- A runtime mesh can carry manually attached provenance (`userData.sourceRef`) and expose it correctly through raycast-driven click interaction.
- Multiple objects generated from the same function maintain independently correct, non-leaking provenance.
- Ordinary React re-renders do not disturb provenance or object identity.
- React key changes force genuine mesh recreation, and provenance is correctly reapplied when this happens, because it is declared in JSX rather than assigned once imperatively.
- A memoized generation pipeline (`useMemo`) revealed that a mesh's *generation-data lifecycle* and its *runtime mesh lifecycle* are independent — `useMemo` governs when generation logic reruns; React Three Fiber's reconciler independently governs whether the underlying `THREE.Mesh` is destroyed and rebuilt, based on JSX element type and key/position, not on whether the data feeding its props changed.
- The `sourceRef` metadata shape (file, function, line, args) required no modification across any of the seven experiments.
- One methodology error (Experiment 5's original HMR test being a hard refresh rather than genuine Hot Module Replacement) was caught, documented, and correctly superseded by Experiment 7's authoritative HMR validation.

Stage 2 does not repeat this evidence. It references it as the basis for the decisions below.

### 3. Stage Objectives

Objective 1 — Lock the `SourceRef` metadata contract.

Objective 2 — Standardize the canonical tagging rule.

Objective 3 — Audit Stage 1's experiments for compliance with the finalized convention.

Objective 4 — Design candidate strategies for future automatic instrumentation, without selecting one.

Objective 5 — Document what this stage intentionally does not decide, to prevent scope creep into later stages.

## 4. Metadata Convention

The metadata contract is frozen for v0.1 as:

```
type SourceRef = {  
  file: string;  
  function: string;  
  line: number;  
  args: Record<string, unknown>;  
  schemaVersion?: number;  
};
```

file, function, and line identify where an object's generating code lives. args captures the exact runtime values used for that specific instance — the property that distinguishes this project from tools that only resolve to a shared line of code regardless of which of many generated instances was clicked.

schemaVersion is included now, unused, as a reserved field: if Stage 6.5's automatic instrumentation eventually generates SourceRef records through a different mechanism than manual tagging, this field allows the two formats to be distinguished without a breaking change to every downstream consumer.

No other fields were added. Stage 1 exercised this exact shape across single objects, multiple generated instances, re-renders, recreation, memoization, and HMR without needing to extend it. Extending a contract that has already been stress-tested and holds is a cost with no offsetting benefit.

The full rationale for this contract, including field-by-field reasoning, lives in docs/architecture/metadata-convention.md — this report summarizes it rather than duplicating it, so the two documents do not drift out of sync as either is revised.

## 5. Canonical Tagging Rule

Provenance must be declared through the JSX userData prop as part of the mesh's declarative definition:

```
<mesh userData={{ sourceRef }}>
```

Provenance must never be attached imperatively after mount — not via a useEffect, a ref callback, or an event handler assigning mesh.userData.sourceRef = ... outside the render path.

This rule is a direct consequence of Stage 1's Experiment 4 and Experiment 7 findings: because a mesh's generation-data lifecycle and runtime lifecycle are independent, a tagging approach must be correct under both "mesh reused, props updated" and "mesh recreated, fresh props applied" without needing to know which case it's in. Declaring userData as a prop satisfies this by construction — it is reapplied on every render regardless of which underlying lifecycle actually changed. An imperative one-time assignment has no such guarantee and can silently fall out of sync the moment either lifecycle fires independently of the other.

## 6. Validation Audit

A desk audit was performed against the existing Stage 1 report to confirm that every experiment, as already documented, complies with the tagging rule above. This was a read-through cross-reference of prior results — not a rebuild, re-run, or new implementation work, consistent with Stage 2's scope as architecture-only.

| Experiment          | Tagging Approach Used | Complies with Rule |
|---------------------|-----------------------|--------------------|
| Exp 1 — Single Cube | JSX userData prop     | Yes                |

| Experiment                  | Tagging Approach Used                                     | Complies with Rule |
|-----------------------------|-----------------------------------------------------------|--------------------|
| Exp 2 — Multiple Cubes      | JSX userData prop, per-instance                           | Yes                |
| Exp 3 — React Re-render     | JSX userData prop                                         | Yes                |
| Exp 4 — Mesh Recreation     | JSX userData prop                                         | Yes                |
| Exp 5 — HMR (superseded)    | JSX userData prop                                         | Yes                |
| Exp 6 — Procedural Trees    | JSX userData prop                                         | Yes                |
| Exp 7 — Memoized Generation | JSX userData prop, tagged inside useMemo at creation time | Yes                |

Observation: every experiment already conformed to the convention as written. This is not a coincidence to be overclaimed — Stage 1's experiments were the evidence *used to derive* the rule in Section 5, so agreement here confirms the rule was extracted correctly from the data rather than asserting something new. No experiment required modification, and none was rerun to produce this table.

## 7. Future Auto-Instrumentation Design

v0.1 relies entirely on the manual tagging rule above. A later stage (6.5, per the project's staged development plan) will replace manual tagging with compile-time instrumentation, once the manual-tagging MVP has been proven in real use. That instrumentation requires an opt-in strategy — a way to know which functions should be instrumented.

Three candidate strategies were identified and compared:

| Strategy                                                | Advantages                                           | Disadvantages                                                                                                                |
|---------------------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Directory include (config-based)                        | Zero developer effort once configured                | Instruments by location, not intent — cannot distinguish a generator function from an unrelated helper in the same directory |
| Comment annotation ( <code>// @click-to-source</code> ) | Explicit, attached to the function itself            | "Soft" explicit — non-semantic, ignored by refactoring tools, can go stale silently                                          |
| Wrapper API ( <code>clickToSource(createTree)</code> )  | Explicit, semantic, discoverable, survives refactors | Adds visible syntax; requires developer adoption of an API surface                                                           |

No strategy is selected at this stage. Selection criteria were documented instead of a choice, weighted so that low false positives and low false negatives are treated as harder constraints than convenience properties such as IDE discoverability — since accurate, per-instance provenance is this project's core differentiator, and a strategy that instruments by the wrong signal would undermine that from the foundation up.

Full detail on this design discussion, including the complete selection-criteria list, lives in `docs/architecture/metadata-convention.md`.

## 8. Architectural Findings

Finding 1. The metadata contract required no changes across every scenario Stage 1 tested, and is frozen for v0.1 as a result.

Finding 2. Declarative (JSX-prop) tagging is the only approach that stays correct across both of R3F's independent lifecycles — generation-data and runtime-mesh — without needing to detect which one fired. Imperative tagging cannot make this guarantee.

Finding 3. Every existing Stage 1 experiment, examined by desk audit, already conforms to the finalized rule. This confirms the rule was derived correctly from evidence rather than introduced as a new requirement the experiments happen to satisfy.

Finding 4. Candidate auto-instrumentation strategies have been identified and evaluated against a weighted set of selection criteria, but none has been implemented or tested — this design targets producing the same SourceRef contract locked in Section 4, not a verified claim that it will preserve identical runtime behavior. That claim remains to be earned at Stage 6.5.

Finding 5. One deliberate gap remains open rather than solved: whether expensive (non-trivial) generation could expose a transient stale-provenance window between an old render's userData and a new render's updated value. Stage 1's cube-based experiments were too fast to expose this. This is documented as a policy — accepted risk for v0.1, to be re-evaluated during Stage 4 dogfooding against real terrain/tree generation — not as an oversight.

## 9. Deliverables

| Deliverable                                                         | Status   |
|---------------------------------------------------------------------|----------|
| Metadata Convention (docs/architecture/metadata-convention.md)      | Complete |
| Canonical Tagging Rule, with rationale                              | Complete |
| Validation Desk Audit (Section 6, this report)                      | Complete |
| Future Instrumentation Design (candidates + criteria, no selection) | Complete |
| Non-Goals / Architectural Boundaries                                | Complete |

## 10. Exit Criteria

- Metadata contract finalized and frozen for v0.1
- Tagging convention standardized, with rationale traceable to Stage 1 evidence
- Existing Stage 1 experiments audited against the convention
- Future instrumentation strategy space documented; selection explicitly deferred
- Stale-provenance timing question documented as an accepted, deferred risk — not silently dropped
- No unresolved architectural blocker identified
- Ready for Stage 3

## 11. Final Assessment

Stage 2 converted Stage 1's experimental findings into a stable architectural contract. A canonical metadata shape and tagging rule were locked, each traceable to specific Stage 1 evidence rather than asserted by convention alone. Existing experiments were audited against that rule by desk review, not by rebuilding anything, consistent with Stage 2's scope as design work rather than implementation. Candidate strategies for future automatic instrumentation were identified and evaluated against explicit, weighted selection criteria

— with the selection itself correctly deferred until there is real implementation experience to decide from, at Stage 6.5.

No runtime code was written during this stage. That restraint is itself part of the deliverable: Stage 3 can now build the core resolution engine and overlay against a contract that will not need to be renegotiated mid-implementation, rather than against assumptions that were never written down.

With the metadata contract, tagging rule, validation audit, and future instrumentation design complete, Click-to-Source 3D is ready to move from architecture into implementation at Stage 3.